

NDAT

Sistemin Konuşlandırılması

Erişim Denetimi (Rol tabanlı erişim denetimi)

Nesnelerin Depolanması

Deployment

- Yazılımın kod ortamından çıkıp, son kullanıcının erişebileceği fiziksel veya bulut tabanlı donanımlara yerleştirilme sürecidir.
- **Neden Önemli?** Tasarladığınız nesnelerin (objelerin) hangi sunucuda yaşadığını, veritabanına nasıl bağlandığını ve güvenliğin nasıl sağlandığını gösterir.
- **Strateji:** "Kodum nerede çalışıyor?" sorusuna yanıt arıyoruz.

UML Deployment Diyagramı

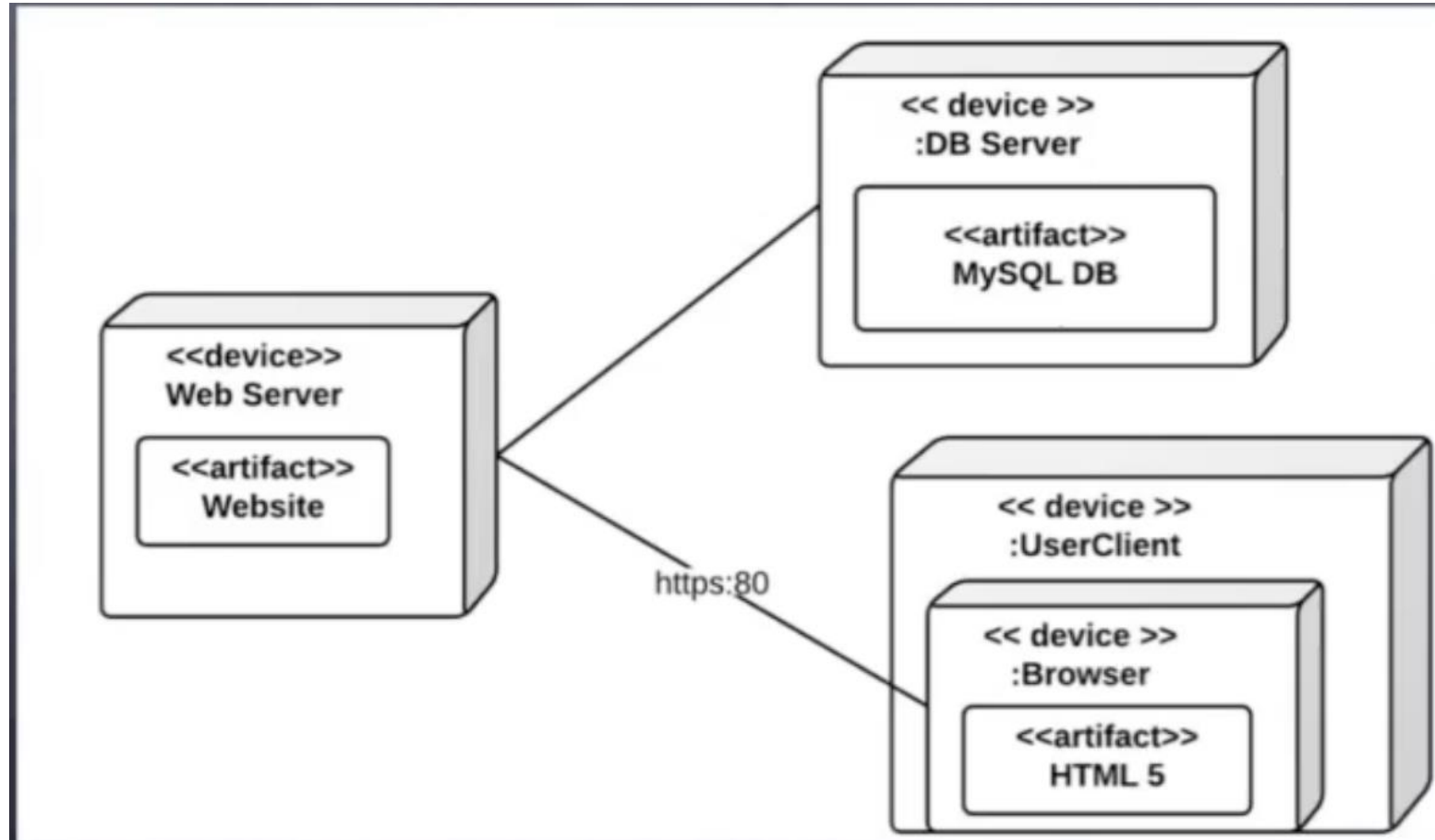
- Yapısal gösterim şekillerindendir.
- Yazılım ürünlerinin/bileşenlerinin düğümlere nasıl yerleştirildiği ile ilgili bilgi verir. Sadece kodun yapısını değil, donanımın yapısını da çizeriz.

Node: Yazılımın üzerinde çalıştığı fiziksel veya sanal donanım (Cihaz , işletim sistemi, sanal makine vb.)

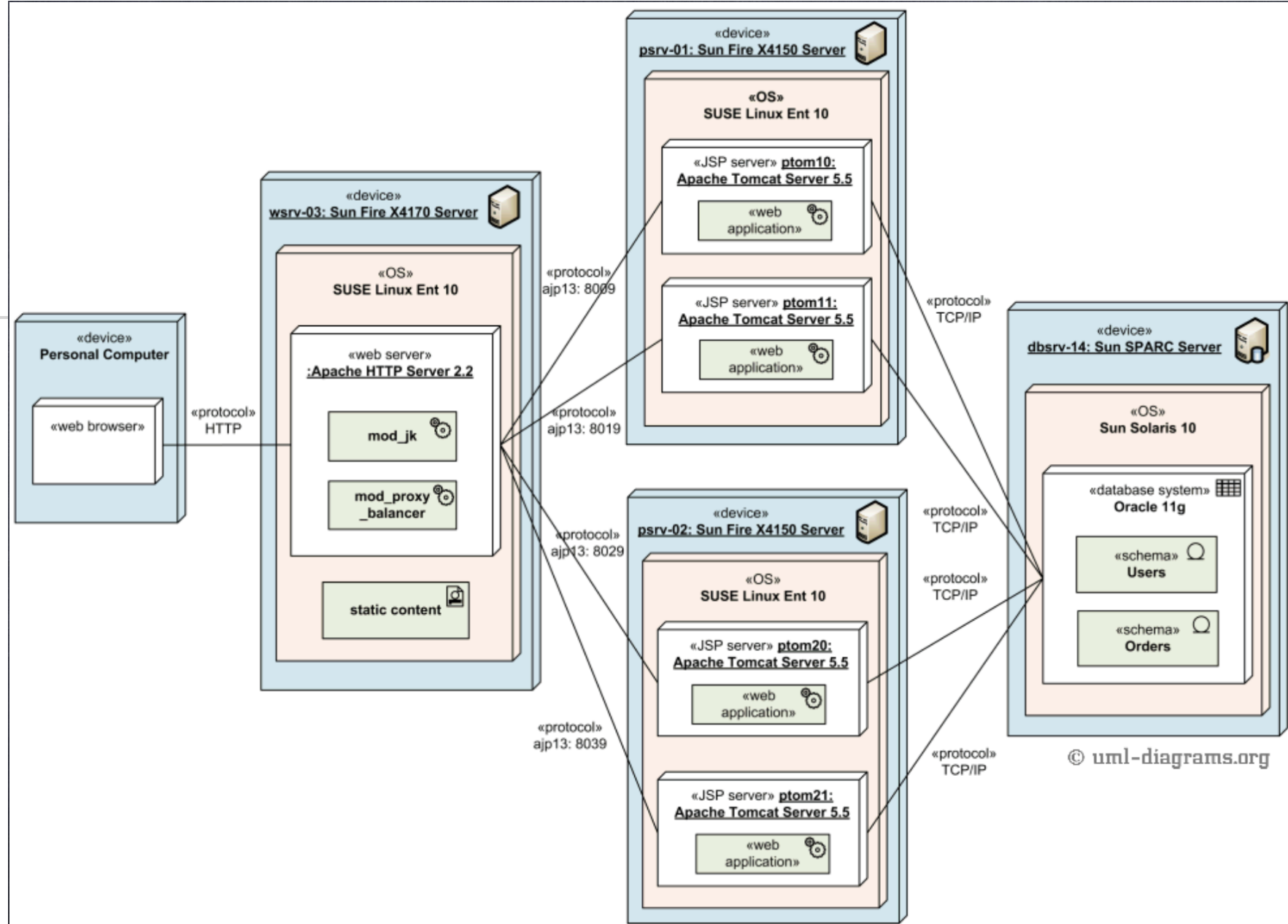
Artifact(Eser): Düğümlere yerleştirilen yazılım parçalarıdır.

Communication path: Düğümler arasındaki bağlantı türünü temsil eder (HTTP/HTTPS, TCP/IP, JDBC).

Sistemin Konuşlandırılması



Sunucu Mimarisi



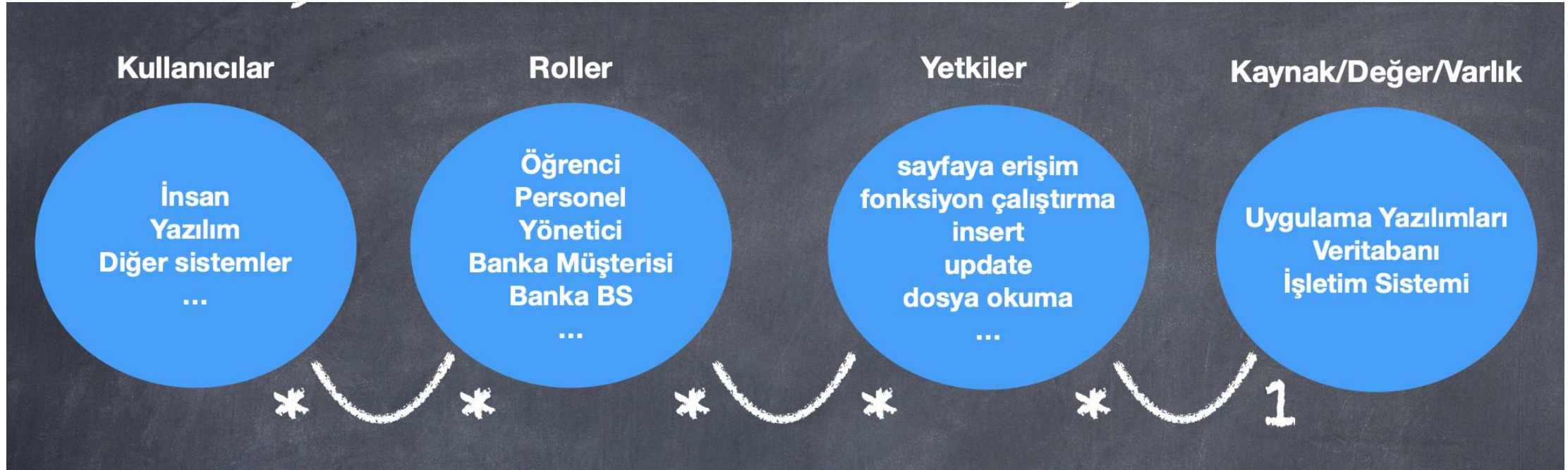
RBAC Nedir?

- Geleneksel yöntemde izinler doğrudan kullanıcılara verilir (örneğin: "Ahmet silebilir"). Ancak kullanıcı sayısı arttığında bu yönetim imkansızlaşır. **RBAC** ile araya bir "Rol" katmanı ekleriz.
- **Prensip:** İzinler rollere atanır, roller ise kullanıcılara.
- **Avantajı:** Bir personelin görevi değiştiğinde sadece rolünü değiştirmeniz yeterlidir; sistemdeki binlerce izni tek tek güncellemenize gerek kalmaz.

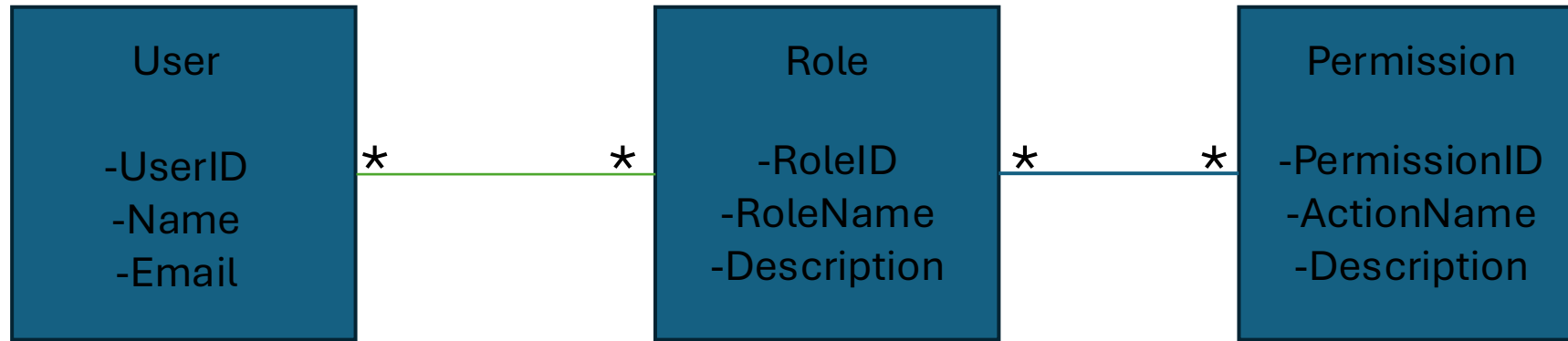
Nesne Yönelimli Model

Bileşen	Açıklama	Örnek
User (Kullanıcı)	Sistemi kullanan kişi.	Ahmet, Ayşe
Role (Rol)	Yetki kümesinin adı.	Admin, Editör, Misafir
Permission (İzin)	Yapılabilen eylem.	Okuma, Yazma, Silme

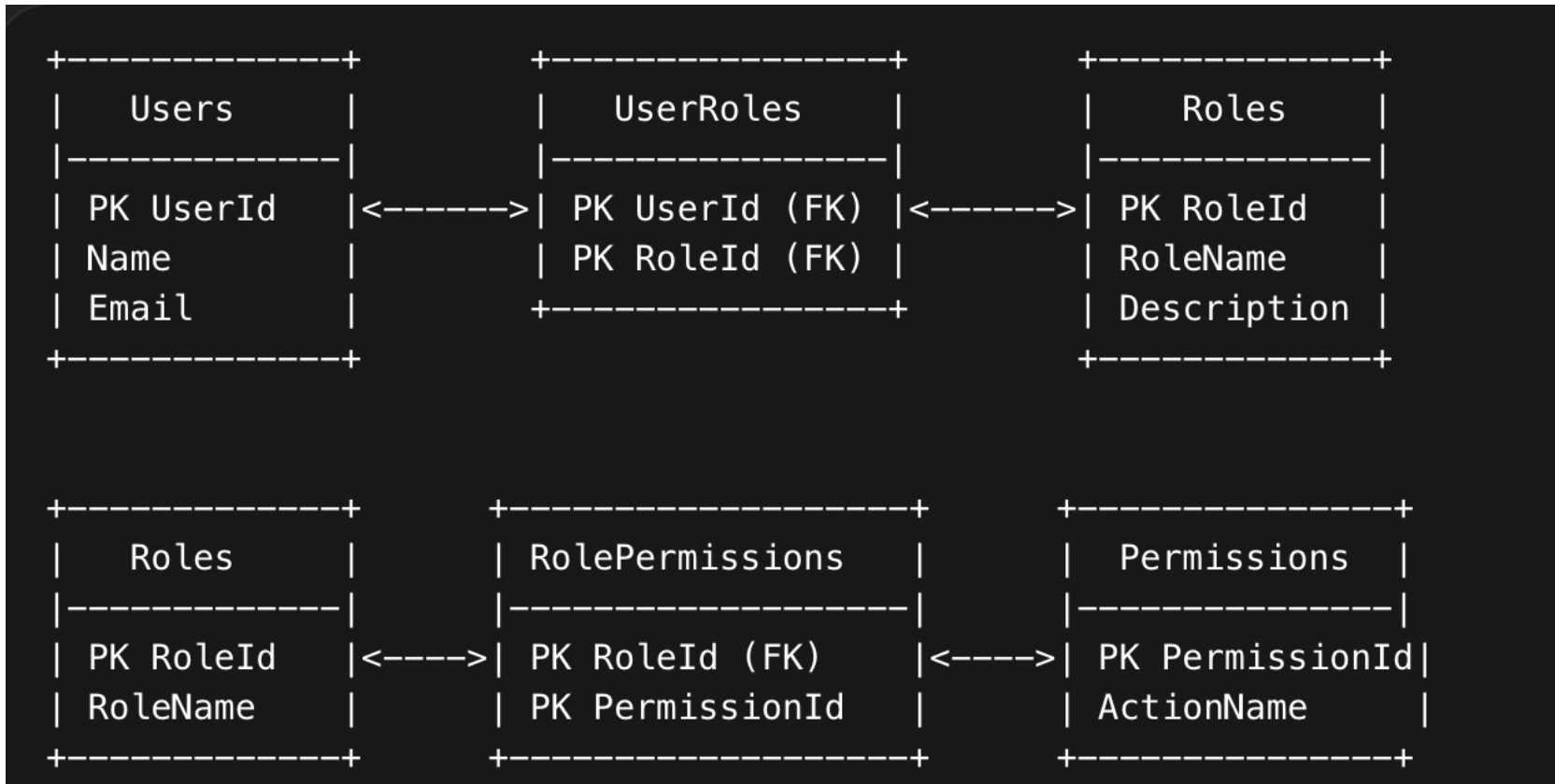
Erişim Denetimi (Rol tabanlı Erişim Denetimi)



RBAC UML Class Diagram



ER (Entity-Relationship) Diyagramı



Sağlık Sistemi

Rol	İzinler (Permissions)
Doktor	Reçete Yaz, Hasta Geçmişi Görüntüle
Hemşire	Vital Bulguları Gir, Hasta Geçmişi Görüntüle
Hasta Bakıcı	Sadece Hasta Listesini Gör
Admin	Kullanıcı Ekle/Sil, Sistem Loglarını Gör

Online Shopping App

ACTIVITY MATRIX				
		External Entities & Roles		
Assests	Action	Guest	Registered User	Admin
Customer Data	Create	Always	Own Data	Always
	Read	Never	Own Data	Always
	Update	Never	Own Data	Always
	Delete	Never	Own Data	Always
Login Page	Access	Always	Always	Always
Customer Profile Page	Access	Never	Always	Never
Login Function	Execute	Always	Always	Always
Add Shopping Chart Function	Execute	Never	Always	Never
Checkout Fuction	Execute	Never	Always	Never

```

@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests() ExpressionUrlAuthorizationConfigurer<H>.ExpressionInterceptUrlRegistry
            .antMatchers("/resources/**", "/").permitAll()
            .antMatchers("/dashboard").hasRole("ADMIN")
            .anyRequest().authenticated()
            .and() HttpSecurity
        .formLogin() FormLoginConfigurer<HttpSecurity>
            .defaultSuccessUrl("/dashboard")
            .and() HttpSecurity
        .logout() LogoutConfigurer<HttpSecurity>
            .permitAll();
}

```

```

@GetMapping("/registeredUser")
@PreAuthorize("hasAuthority('RegisteredUser')")
public String moderatorAccess() {
    return "RegisteredUser Board.";
}

@GetMapping("/administrator")
@PreAuthorize("hasAuthority('Administrator')")
public String adminAccess() { return "Admin Board."; }

```

Yetkilendirme İşlemleri

PUBLIC: Tüm roller / kullanıcılar.

kullanıcıAdı: Tek bir kullanıcı.

ALL: Tüm yetkiler.

- rol1 isimli role customers tablosu üzerinde seçim yapma yetkisi ver.

```
GRANT SELECT ON "customers" TO "rol1";
```

- Tüm rollere customers tablosu üzerinde kayıt ekleme yetkisi ver.

```
GRANT INSERT ON "customers" TO PUBLIC;
```

- rol1 isimli kullanıcıya customers tablosu üzerinde tüm yetkileri ver.

```
GRANT ALL ON "customers" TO "rol1";
```

- rol1 isimli rolün customers tablosu üzerindeki güncelleme yetkisini geri al.

```
REVOKE UPDATE ON "customers" FROM "rol1";
```

- rol1 isimli rolün customers tablosu üzerindeki tüm yetkilerini geri al.

```
REVOKE ALL ON "customers" FROM "rol1";
```

- rol1 kullanıcısının Sema1 içerisindeki nesnelere ait tüm yetkileri geri alınır.

```
REVOKE ALL ON SCHEMA "Sema1" FROM "rol1";
```

MVC (Model-View-Controller)

- **Model (Veri ve Mantık)**

Uygulamanın kalbidir. Verinin nasıl saklanacağını, kurallarını ve iş mantığını (Business Logic) içerir.

- **Görev:** Veritabanı işlemlerini yönetmek, hesaplamaları yapmak.
- **Restoran Benzetmesi:** Mutfağın kendisidir. Yemeklerin tarifleri (mantık) ve malzemeler (veri) burada bulunur.

- **View (Görünüm/Arayüz)**

Kullanıcının gördüğü ve etkileşime girdiği her şeydir (HTML, CSS, JSON).

- **Görev:** Model'den gelen veriyi kullanıcıya şık ve anlaşılır bir şekilde sunmak.
- **Restoran Benzetmesi:** Menü kartı veya tabağın sunum şeklidir. Müşteri (kullanıcı) yemeği buradan seçer ve görür.

- **Controller (Yönetici/Aracı)**

Kullanıcıdan gelen istekleri karşılayan ve Model ile View arasındaki trafiği yöneten katmandır.

- **Görev:** Gelen isteği analiz eder, gerekiyorsa Model'e veri işletir ve hangi View'un gösterileceğine karar verir.
- **Restoran Benzetmesi:** Garsondur. Müşteriden siparişi alır (İstek), mutfağa iletir (Model) ve hazır olan yemeği müşteriye sunar (View).

MVC Çalışma Akışı

Kullanıcı bir butona tıkladığında neler olur?

- 1.İstek: Kullanıcı tarayıcı üzerinden bir URL'ye (örn: /Urun/Detay/5) istek gönderir.
2. Yönlendirme (Routing): Sistem bu isteği ilgili Controller'a (UrunController) yönlendirir.
3. İşlem: Controller, 5 ID'li ürünün bilgilerini almak için Model'e danışır.
4. Veri Hazırlığı: Model, veritabanından veriyi çeker ve Controller'a geri döner.
5. Yanıt (Response): Controller, bu veriyi alır ve View klasöründeki ilgili şablona (Detay.cshtml) gönderir. Sonuç kullanıcıya HTML olarak döner.

Neden MVC?

- Gevşek Bağlılık (Loose Coupling):** Görünüm (View) değiştiğinde veritabanı koduna (Model) dokunmanız gerekmez.
- Yeniden Kullanılabilirlik:** Aynı Model yapısını hem bir web sitesi hem de bir mobil uygulama API'si için kullanabilirsiniz.
- Test Edilebilirlik:** Arayüzden bağımsız olarak iş mantığını (Model ve Controller) otomatik testlere (Unit Test) tabi tutabilirsiniz.
- Paralel Geliştirme:** Bir ekip arayüzü (View) tasarlarken, diğer ekip arka plan mantığını (Model) aynı anda kodlayabilir.

MVC Uygulama

// MODEL

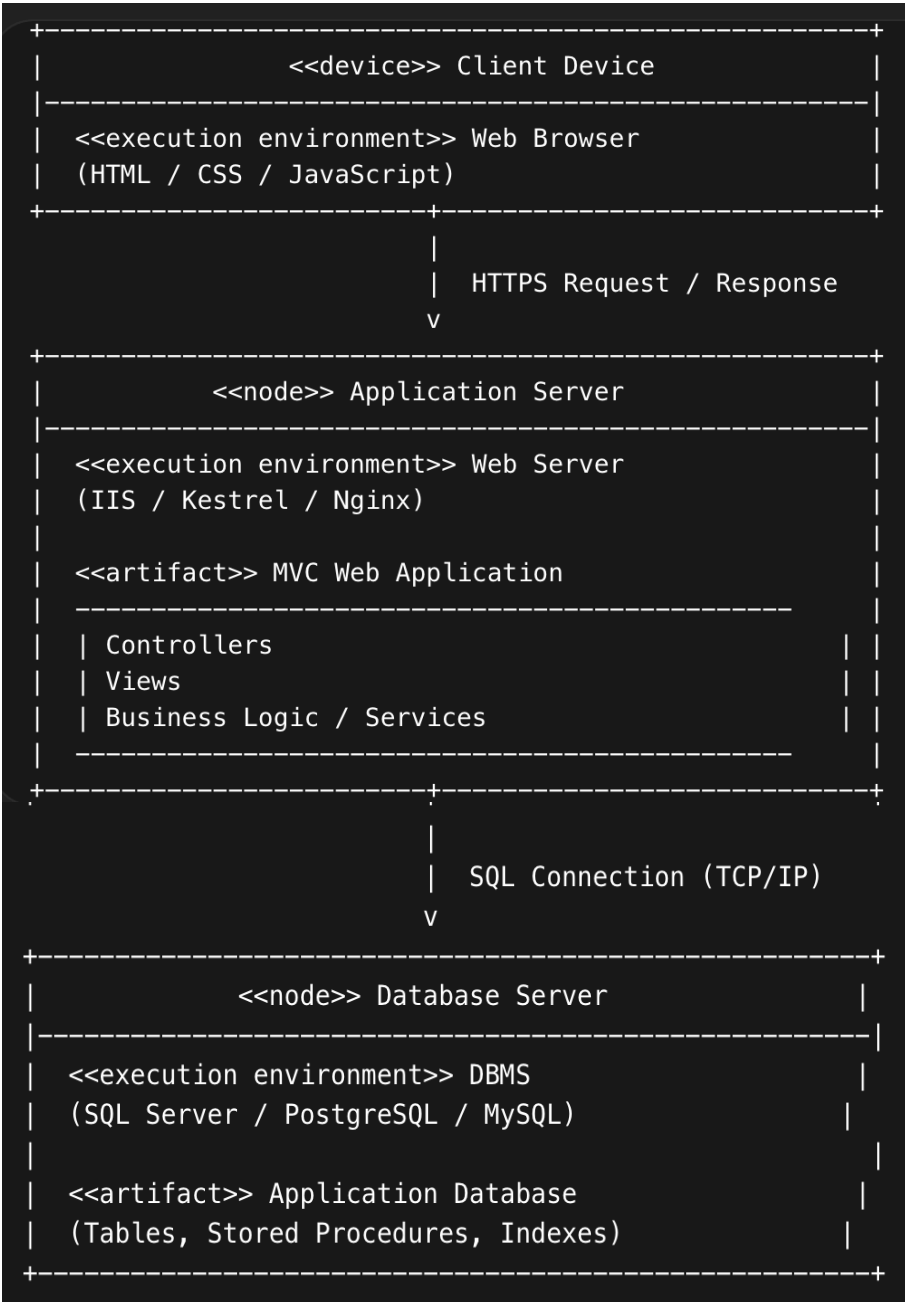
```
public class MesajModel {  
    public string Metin { get; set; }  
}
```

// CONTROLLER

```
public class MerhabaController : Controller {  
    public IActionResult Index() {  
        var model = new MesajModel { Metin = "Merhaba, MVC Dünyasına Hoş Geldiniz!" };  
        return View(model); // Modeli View'a gönder  
    }  
}
```

// VIEW (Index.cshtml)

```
<h1> @Model.Metin </h1>
```



A. Sunum Katmanı (Client Side)

Cihaz: Kullanıcının tarayıcısı (Chrome, Safari).

İşlem: Sadece HTML/JS render edilir.

B. Uygulama Katmanı (Application Server)

Node: Web Sunucusu (IIS, Kestrel veya Nginx).

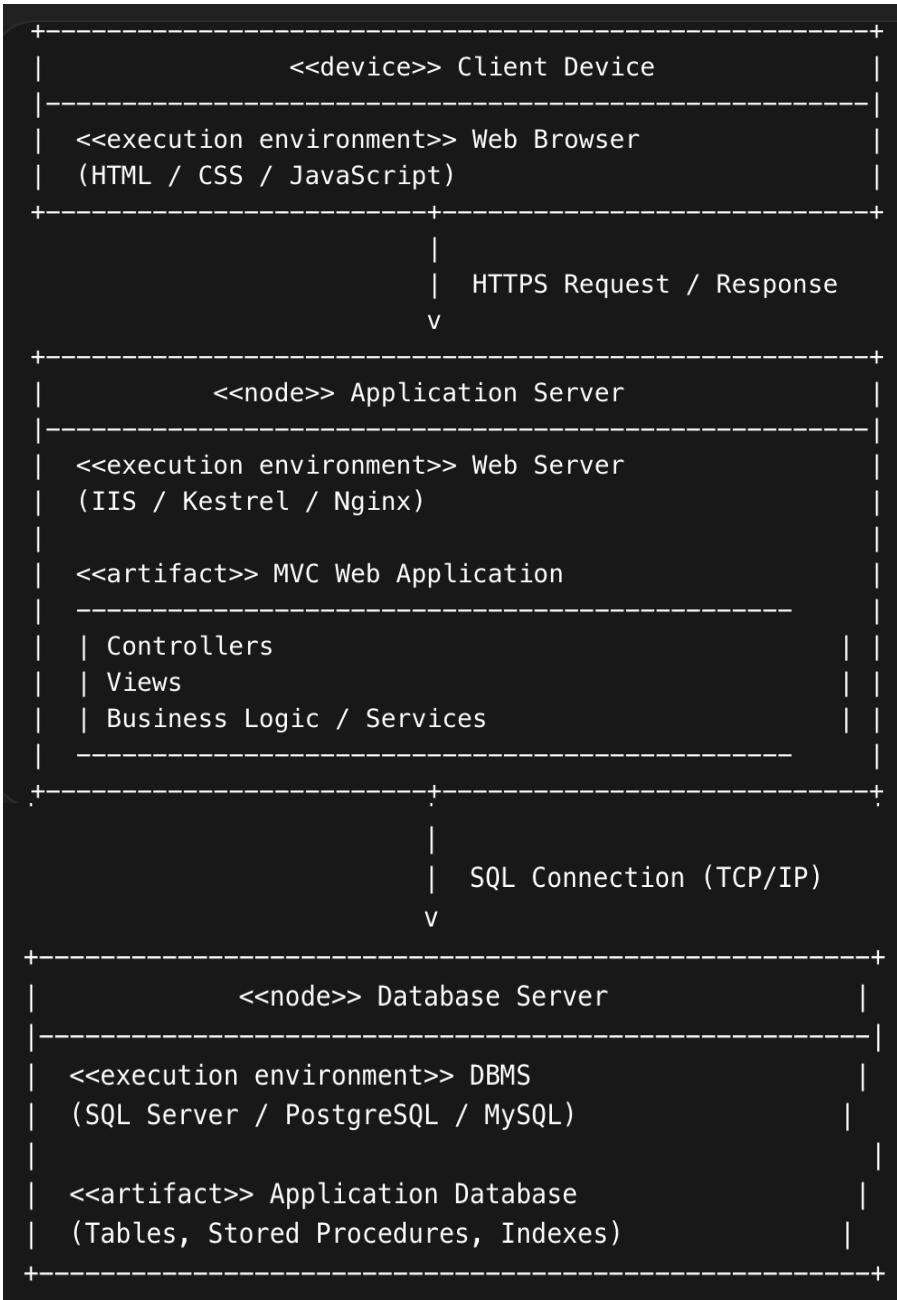
Artifact: Derlenmiş MVC projeniz (Controller ve Logic dosyaları).

Görev: İstekleri karşılar, iş mantığını çalıştırır.

C. Veri Katmanı (Database Server)

Node: SQL Server, PostgreSQL veya MySQL yüklü sunucu.

İletişim: Uygulama katmanı ile SQL bağlantısı (bağlantı dizgisi/connection string) üzerinden konuşur.



A. Sunum Katmanı (Client Side)

Cihaz: Kullanıcının tarayıcısı (Chrome, Safari).

İşlem: Sadece HTML/JS render edilir.

B. Uygulama Katmanı (Application Server)

Node: Web Sunucusu (IIS, Kestrel veya Nginx).

Artifact: Derlenmiş MVC projeniz (Controller ve Logic dosyaları).

Görev: İstekleri karşılar, iş mantığını çalıştırır.

C. Veri Katmanı (Database Server)

Node: SQL Server, PostgreSQL veya MySQL yüklü sunucu.

İletişim: Uygulama katmanı ile SQL bağlantısı (bağlantı dizgisi/connection string) üzerinden konuşur.

MVC Yetkilendirme

```
// Bu nesne sadece 'Doktor' rolüyle erişilebilir
[Authorize(Roles = "Doktor")]
public class ReceteController : Controller
{
    public IActionResult Yaz(ReceteModel model)
    {
        // İş mantığı (Business Logic)
        return View("Basarili");
    }
}
```

```
// Sadece 'Admin' rolüne sahip olanlar bu metodu
çalıştırabilir
[Authorize(Roles = "Admin")]
public IActionResult KullaniciSil(int id)
{
    // Silme mantığı burada çalışır
    return RedirectToAction("Index");
}

// Hem 'Admin' hem 'Uye' rolü buraya erişebilir
[Authorize(Roles = "Admin, Uye")]
public IActionResult KitapListele()
{
    return View();
}
```

Nesnelerin Depolanması

- **"Nesnelerin Depolanması" (Object Persistence)**, bellekte yaşayan geçici nesnelerin, uygulama kapandıktan sonra da varlığını sürdürebilmesi için kalıcı bir ortama (genellikle ilişkisel bir veritabanına) aktarılması sürecidir.

Transient(Geçici) Nesne: Sadece çalışma zamanında RAM'de olan nesne.

Persistent (Kalıcı) Nesne: Durumu veritabanına kaydedilmiş ve ihtiyaç duyulduğunda geri yüklenebilen nesne.

Nesne-İlişkisel Empedans Uyumsuzluğu

- Nesne yönelimli dünya ile ilişkisel veritabanı (RDBMS) dünyası farklı diller konuşur. Bu farka "**Impedance Mismatch**" denir:

Nesne Dünyası (C#, Java)

Sınıflar (Classes)

Nesneler (Objects)

Özellikler (Properties)

Kalıtım (Inheritance)

Referanslar (Associations)

İlişkisel Dünya (SQL Server, MySQL)

Tablolar (Tables)

Satırlar (Rows)

Sütunlar (Columns)

SQL'de doğrudan karşılığı yok!

Foreign Key (Yabancı Anahtar)

Katmanlı Mimari ve Depolama

- Nesneleri depolarken doğrudan "Insert/Update" sorguları yazmak yerine, NDAT prensipleri gereği **Data Access Layer (Veri Erişim Katmanı)** kullanılır.

ORM (Object-Relational Mapping)

- Nesneler ile tablolar arasındaki köprüdür.
- **Popüler Araçlar:** Entity Framework (C#), Hibernate (Java).
- **Nasıl Çalışır?** Siz bir Musteri nesnesini kaydedersiniz, ORM aracı bunu otomatik olarak INSERT INTO Musteriler... sorgusuna dönüştürür.

ORM olmadan önce?

- ORM kullanmadığınızda, kodunuzun içine SQL sorguları gömmeniz gerekir. Bu durum, nesne yönelimli yapıya aykırıdır çünkü "String" ifadelerle veritabanı yönetmeye çalışırsınız.
- **Sorun:** SQL'de bir hata yaparsanız bunu ancak uygulama çalışırken (Runtime) fark edersiniz. Ayrıca veritabanı değişirse tüm kodları elle güncellemeniz gerekir.

ORM Yaklaşımı Örneği (Öğrenci Kaydı)

```
public class Ogresnci {  
    public int Id { get; set; }  
    public string Ad { get; set; }  
    public int Numara { get; set; }  
}
```

SQL ile (ORM Yok): INSERT INTO Ogresnciler (Ad, Numara) VALUES ('Ahmet', 123)

ORM ile (Örn: Entity Framework):

```
var yeniOgresnci = new Ogresnci { Ad = "Ahmet", Numara = 123 };  
db.Ogresnciler.Add(yeniOgresnci); // Sadece nesneyi listeye ekler gibi eklersiniz.  
db.SaveChanges(); // ORM arka planda SQL'i kendisi oluşturur ve çalıştırır.
```

Bir ilişki örneği

- Bir Kurs nesnesinin içinde o kursa kayıtlı Öğrenci listesi olabilir:

```
public class Kurs {  
    public string KursAdi { get; set; }  
    public List<Öğrenci> KayitliÖğrenciler { get; set; } // Bu bir Navigation Property'dir.  
}
```

ORM, yazılımcının veritabanı gibi düşünmesini zorunlu kılmak yerine, veritabanının yazılımcı gibi (nesnelerle) konuşmasını sağlar.

ORM 'nin Avantajları

- Tip Güvenliği (Type Safety):** SQL yazarken kolon ismini yanlış yazabilirsiniz ama kod yazarken
- IDE (Visual Studio vb.) size hatalı bir özellik yazdığınızda anında uyarı verir.
- Veritabanı Bağımsızlığı:** Uygulamanızı SQL Server'dan PostgreSQL'e taşımak isterseniz, tek bir ayar satırını değiştirmeniz yeterlidir. ORM, yeni veritabanının diline (SQL lehçesine) otomatik adapte olur.
- Hızlı Geliştirme:** CRUD (Create, Read, Update, Delete) işlemleri için yüzlerce satır SQL yazmak yerine basit metodlar kullanılır.